

Tarefa 14: Comunicação Ponto-a-Ponto em MPI

Latência e largura de banda medidas com ping-pong no supercomputador do NPAD

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Setembro de 2026

1. Objetivo

Implementar um programa MPI com exatamente dois processos, em que o processo 0 envia uma mensagem ao processo 1 e este a devolve imediatamente (ping-pong). Medir com `MPI_Wtime` o tempo de várias trocas consecutivas para tamanhos de 8 bytes a 16 MB, analisar graficamente o tempo em função do tamanho e identificar os regimes dominados pela latência e pela largura de banda.

2. Ambiente de Execução

Item	Valor
Máquina	Supercomputador NPAD/UFRN, partição amd-512, nós exclusivos
Nós	r2n03 e r2n04 — um processo por nó (a mensagem atravessa a rede)
Processador	2 × AMD EPYC 7713 (Milan) por nó
MPI	OpenMPI 5.0.10 (libraries/openmpi/5.0.10-gnu14-ucxmt)
Compilador	GCC 14.2.0 via mpicc -O2
Job	SLURM 2117758

3. Metodologia

Uma **troca** é uma ida (`MPI_Send` de 0 para 1) e uma volta (`MPI_Send` de 1 para 0), cada uma casada com um `MPI_Recv`. Para cada tamanho, o programa faz 10 trocas de aquecimento (que também sincronizam os dois processos) e mede o tempo total de 10 000 trocas (até 64 KB) ou 200 trocas (acima disso). O tempo de ida é metade do tempo por troca, e a banda efetiva é $2n / \text{tempo por troca}$.

Os dados são comparados ao modelo linear de comunicação $t(n) = L + n/B$, com a latência L estimada pela menor mensagem (8 B) e a banda B pela maior (16 MB).

4. Resultados

Tamanho	Trocas	Tempo total (s)	Tempo de ida (μs)	Banda efetiva (GB/s)
8 B	10000	0,0254	1,27	0,006
16 B	10000	0,0265	1,33	0,012
32 B	10000	0,0288	1,44	0,022
64 B	10000	0,0312	1,56	0,041
128 B	10000	0,0309	1,54	0,083
256 B	10000	0,0355	1,77	0,144
512 B	10000	0,0357	1,78	0,287
1 KB	10000	0,0398	1,99	0,514
2 KB	10000	0,0505	2,53	0,810

Tamanho	Trocas	Tempo total (s)	Tempo de ida (μs)	Banda efetiva (GB/s)
4 KB	10000	0,0626	3,13	1,309
8 KB	10000	0,0758	3,79	2,162
16 KB	10000	0,0980	4,90	3,344
32 KB	10000	0,1314	6,57	4,988
64 KB	10000	0,1882	9,41	6,965
128 KB	200	0,0066	16,59	7,903
256 KB	200	0,0116	29,00	9,040
512 KB	200	0,0191	47,84	10,959
1 MB	200	0,0361	90,29	11,613
2 MB	200	0,0701	175,23	11,968
4 MB	200	0,1381	345,13	12,153
8 MB	200	0,2743	685,68	12,234
16 MB	200	0,5457	1.364,36	12,297

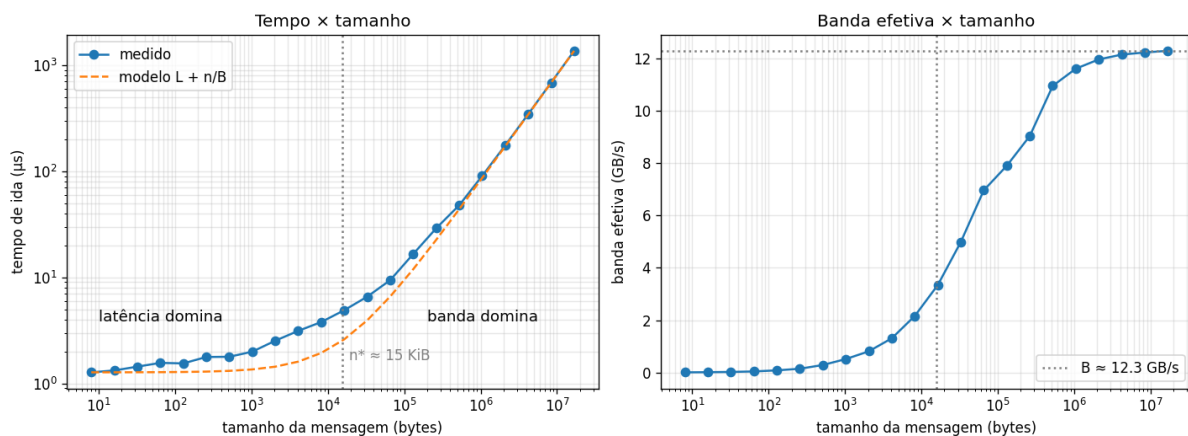


Figura 1 — Tempo de ida (esquerda, escala log-log) e banda efetiva (direita) em função do tamanho da mensagem.

5. Análise

Parâmetro	Valor	Significado
Latência L	1,27 μs	Custo fixo de qualquer mensagem, mesmo vazia
Banda B	12,3 GB/s	Taxa máxima de transferência entre os nós
Corte $n^* = L \cdot B$	≈ 15 KB	Tamanho em que latência e transferência pesam igual

Regime de latência (até ~1 KB). O tempo de ida fica praticamente constante, entre 1,3 e 2 μs, enquanto o tamanho cresce 128×. Enviar 8 B ou 512 B custa quase o mesmo e a banda efetiva é ínfima. Nesse regime, a forma de ganhar desempenho é enviar *menos* mensagens, agrupando dados pequenos em uma só.

Transição (~1 KB a ~1 MB). O tempo passa a crescer com n e a banda efetiva sobe rapidamente, atingindo metade do máximo por volta de 64 KB. Nessa faixa o tempo medido fica acima do modelo linear, por custos que ele não representa: cópias para buffers internos e a mudança do protocolo *eager* para *rendezvous*, que acrescenta um handshake antes de mensagens grandes.

Regime de largura de banda (acima de ~1 MB). O tempo cresce linearmente com n — dobrar a mensagem dobra o tempo — e a banda efetiva se estabiliza em ≈ 12,3 GB/s, compatível com um enlace InfiniBand de 100 Gb/s. A latência fixa representa menos de 1% do tempo a partir de 1 MB; aqui só reduzir o volume de dados enviados melhora o desempenho.

6. Conclusão

A comunicação ponto-a-ponto entre dois nós do NPAD é descrita bem pelo modelo $t(n) = L + n/B$, com $L \approx 1,3 \mu s$ e $B \approx 12,3 \text{ GB/s}$. Mensagens menores que $\sim 15 \text{ KB}$ são dominadas pela latência e maiores que isso pela largura de banda. Para programas MPI, isso significa que muitas mensagens pequenas desperdiçam a rede e que vale agregar dados antes de comunicar.

7. Código-Fonte

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  /* Uma troca = ida (0 -> 1) + volta (1 -> 0). */
6  static void troca(char *buf, int n, int rank) {
7      if (rank == 0) {
8          MPI_Send(buf, n, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
9          MPI_Recv(buf, n, MPI_CHAR, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
10     } else {
11         MPI_Recv(buf, n, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
12         MPI_Send(buf, n, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
13     }
14 }
15
16 int main(int argc, char **argv) {
17     int rank, size;
18     MPI_Init(&argc, &argv);
19     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
20     MPI_Comm_size(MPI_COMM_WORLD, &size);
21     if (size != 2) {
22         if (rank == 0) fprintf(stderr, "Use exatamente 2 processos.\n");
23         MPI_Finalize();
24         return 1;
25     }
26
27     const int max = 16 << 20; /* 8 B até 16 MB */
28     char *buf = calloc(max, 1);
29     if (rank == 0) printf("bytes,trocas,tempo_total_s,tempo_por_troca_us,banda_MBps\n");
30
31     for (int n = 8; n <= max; n *= 2) {
32         int trocas = n <= 65536 ? 10000 : 200;
33
34         for (int i = 0; i < 10; i++) troca(buf, n, rank); /* aquecimento */
35
36         double t0 = MPI_Wtime();
37         for (int i = 0; i < trocas; i++) troca(buf, n, rank);
38         double t = MPI_Wtime() - t0;
39
40         if (rank == 0) {
41             double rtt = t / trocas; /* ida e volta */
42             printf("%d,%d,%.6f,%.3f,%.1f\n", n, trocas, t, rtt * 1e6,
43                 2.0 * n / rtt / 1e6); /* 2n bytes por troca */
44         }
45     }
46     free(buf);
47     MPI_Finalize();
48     return 0;
49 }
50
```

Figura 2 — pingpong.c.